

[14X030] Introduction to Computational Finance

Exercise # 9

April 28, 2026

General instructions

Each student is expected to upload on Moodle a **zip** file named as *firstname_lastname*, containing the followings:

- A report in **pdf** format to answer exercise questions.
- The **code** used to generate the results of the report.
- If preferred, submit a unique **Jupyter notebook** with both code and report explanations.

Deeper instructions

- **Report:**
 - Answers to the TP questions. Figures and numerical results are necessary but **strictly not** sufficient: provide your analysis/comments as well.
 - Example: if your results surprise you, explain why and explain what the expected result was.
 - **Pay attention** to the presentation: axis labels, legend, etc.
- **Code:**
 - Implementation questions + code used to produce the results and figures in the report.
 - Programming language of your choice.
 - Do not use ChatGPT, or any other LLM/AI-assisted tools while writing down your code. Try by yourself, otherwise it will be counted as failed.
 - I **strongly recommend Python/Julia**: I won't be able to assist you in the same way if you use a different language.

Deadline

Any questions to: [Lorenzo Bini](#).

Upload on Moodle due by : **May 4, 2026 at 11:59 pm**

Option pricing: Black-Scholes versus Binomial Tree

Let an asset S be valued at $t = 0$ at $S_0 = 100$. We consider an european option to buy this asset (call) with maturity $T = 1$ (in years) and strike price $K = 120$. The goal of this exercise is to compare two methods to price this option, Black-Scholes and Binomial Tree. We assume a constant volatility $\sigma = 20\%$ over the lifespan of the call, and a risk-free rate $r = 5\%$.

1. Implement the Black-Scholes formula to determine the value of this call at $t = 0$.
2. Implement a binomial tree to determine the initial value of the call. Your implementation should take the depth of the tree as an argument.
3. On the same graph, plot the evolution of the estimated value of the call option as a function of the binomial tree depth, as well as the value derived with Black-Scholes. What do you observe? How deep should be the tree in order to get a reasonable approximation of the Black-Scholes value?

Implied Volatility from Binomial Prices

Using your binomial-tree pricer, compute the implied Black-Scholes volatility for different strikes and tree depths. Plot the resulting volatility “smile” and discuss convergence as the tree deepens.

1. Strikes & Tree Depths:

- Fix $S_0 = 100$, $T = 1$, $r = 0.05$.
- Consider strikes $K \in \{80, 90, 100, 110, 120\}$.
- Use three binomial-tree depths: $N \in \{20, 100, 500\}$.

2. Compute Tree Prices: For each (K, N) , compute the call price

$$C_{\text{tree}} = \text{binomial_call}(S_0, K, T, r, \sigma_{\text{true}}, N), \quad \sigma_{\text{true}} = 0.20.$$

3. Implied Volatility via Bisection Method:

- Implement the Black-Scholes call price

$$C_{\text{BS}}(S_0, K, T, r, \sigma) = S_0 N(d_1) - K e^{-rT} N(d_2),$$

$$\text{with } d_{1,2} = \frac{\ln(S_0/K) + (r \pm \frac{1}{2}\sigma^2)T}{\sigma\sqrt{T}}.$$

- For each tree price C_{tree} , solve for σ_{imp} satisfying

$$C_{\text{BS}}(S_0, K, T, r, \sigma_{\text{imp}}) = C_{\text{tree}}$$

by using a bisection ¹ over $\sigma \in [10^{-4}, 2.0]$ with tolerance 10^{-6} .

¹Instead of coding the bisection loop yourself, you can use Python's `scipy.optimize.bisect`. Define $f(\sigma) = C_{\text{BS}}(S_0, K, T, r, \sigma) - C_{\text{tree}}$. Call `bisect(f, 1e-4, 2.0, xtol=1e-6)` to find σ_{imp} . This will be more concise and handles convergence for you.

4. Volatility Smile Plot:

- On one chart, plot $\sigma_{\text{imp}}(K)$ vs. K for each tree depth N , then add an horizontal line at $\sigma_{\text{true}} = 0.20$ as well.
- Comment on how the smile flattens as N increases.